

SYSTEM ARCHITECTURE DOCUMENT

Membership DBMS, Financial Management and Reporting Modules

Comprehensive Automation of KSM Operations

Prepared for: Software Development Team, Database Architects and Testers

Prepared by: PN Consulting - System Analysis and Systems Consulting

September 2026

Confidential — Internal / Development Team Use Only

Table of Contents

Contents

Table of Contents	2
1. Purpose, Scope and Audience	4
1.1 Scope	4
1.2 Audience — Read This Document As:	4
2. Architecture Style: Clean Architecture on .NET	5
2.1 Why Clean Architecture for This System	5
2.2 The Dependency Rule	5
2.3 The Four Layers, at a Glance	6
2.4 Technical Principles	6
3. Solution and Project Structure	8
3.1 Reference Rules — Enforced, Not Just Documented	9
3.2 Illustrative Folder Layout	9
5. Repository Pattern and Unit of Work.....	10
5.1 The Pattern, Concretely	10
7. System Context	11
Actors.....	12
8. Data Architecture	12
8.1 Schema Strategy.....	12
8.2 Core Entities (Domain Layer)	13
8.3 Data Integrity Rules — Enforced in the Domain and Application Layers.....	14
8.6 Data Retention	14
9. Cross-Cutting Concerns	15
9.1 MediatR Pipeline Behaviours.....	15
9.2 Approval Workflow	15
9.3 Notification and Document Handling.....	15
10. Security Architecture	16
10.1 Authentication and Authorisation.....	16
10.2 Data Protection	16
10.3 Application Security (OWASP Top 10)	16
10.4 Audit Logging.....	17
10.5 Segregation of Duties.....	17
11. Integration Architecture	18
11.1 API Design.....	18

11.2 Payment Gateway Adapter.....	18
11.3 Notification Adapter	18
11.4 Future Integration API (DBTF, RECCO, Website, Mobile App).....	18
12. Deployment / Infrastructure Architecture	20
12.1 Environment Strategy	20
12.2 Sizing (BRD #15, restated for the development team)	21
12.3 Backup, Restore and Disaster Recovery	21
13. Key Workflow Architecture	22
Other BRD Workflows, and Where They Live	22
14. Testing Strategy.....	24
14.1 Test Projects and What Each One Covers.....	24
14.2 What to Test at Each Layer	24
14.3 Mocking Strategy — Why Testers Should Care About the Repository Pattern	25
14.4 UAT and BRD Acceptance Criteria	25
14.5 CI Gate	25
16. Development Roadmap and Build Order	27
16.1 Before Any Feature Work — Inception Tasks	27
16.3 Phase-by-Phase Technical Delivery	27
18. Architecture Decision Log	29
19. Appendix.....	31
A. Requirement Traceability	31
B. Glossary	31
C. Reference Documents	32

1. Purpose, Scope and Audience

This document translates the approved Business Requirements Document (BRD v1.0, July 2026) for the Membership Database Management, Financial Management, and Reporting modules into a concrete, buildable technical architecture: **Clean Architecture on C# / .NET, using the Repository Pattern.**

The existing membership system is roughly 60% complete, built on PHP against a MySQL/MariaDB database. This document does really not extend that codebase in place, however the backend is rebuilt on .NET, structured as described in Sections 2-6, while the existing system's data model, business rules and the gaps already identified against the BRD's Membership requirements remain the direct input to what the new Domain and Application layers must reproduce and complete.

1.1 Scope

- **Membership Database Management** — FR-MEM-001 to FR-MEM-020 (BRD #4.1): member records, recruitment/registration, degree progression, status lifecycle and approvals, transfers, duplicate detection, bulk import/migration.
- **Financial Management** — BRD #4.2: chart of accounts, general ledger, cash book, bank reconciliation, budgeting, payment capture (cash, bank transfer, gateway), and configurable approval workflow.
- **Reporting** — BRD #4.3: role-scoped dashboards, the standard report library, ad-hoc report builder, export and scheduled delivery.
- DBTF, RECCO, the public website and a future mobile app are out of scope for build, but this architecture's API surface (Section 11.4) is designed so they can integrate later without change here.

1.2 Audience — Read This Document As:

- **Software Development Team** — Sections 2–6, 9–11, 13 and 16 are your primary build reference: the layers, the solution structure, the Repository Pattern, cross-cutting concerns, integrations, workflow, and build order. Section 18 (ADR log) is your quick-reference for "why was it built this way."
- **Database Architects** — Section 8 (Data Architecture) is written for you specifically: schema strategy, the entity model, integrity rules, migration strategy, and how EF Core migrations are governed. Section 5 (Repository Pattern) and Section 4.3 (Infrastructure Layer) describe how your schema is reached from code — read them before finalising physical design.
- **Testers** — Section 14 (Testing Strategy) is built around this architecture specifically — what to test at each layer, how the Repository Pattern makes Application-layer logic testable without a database, and the test project structure. Section 13's workflow and Section 6's module map tell you what a feature touches end-to-end.

2. Architecture Style: Clean Architecture on .NET

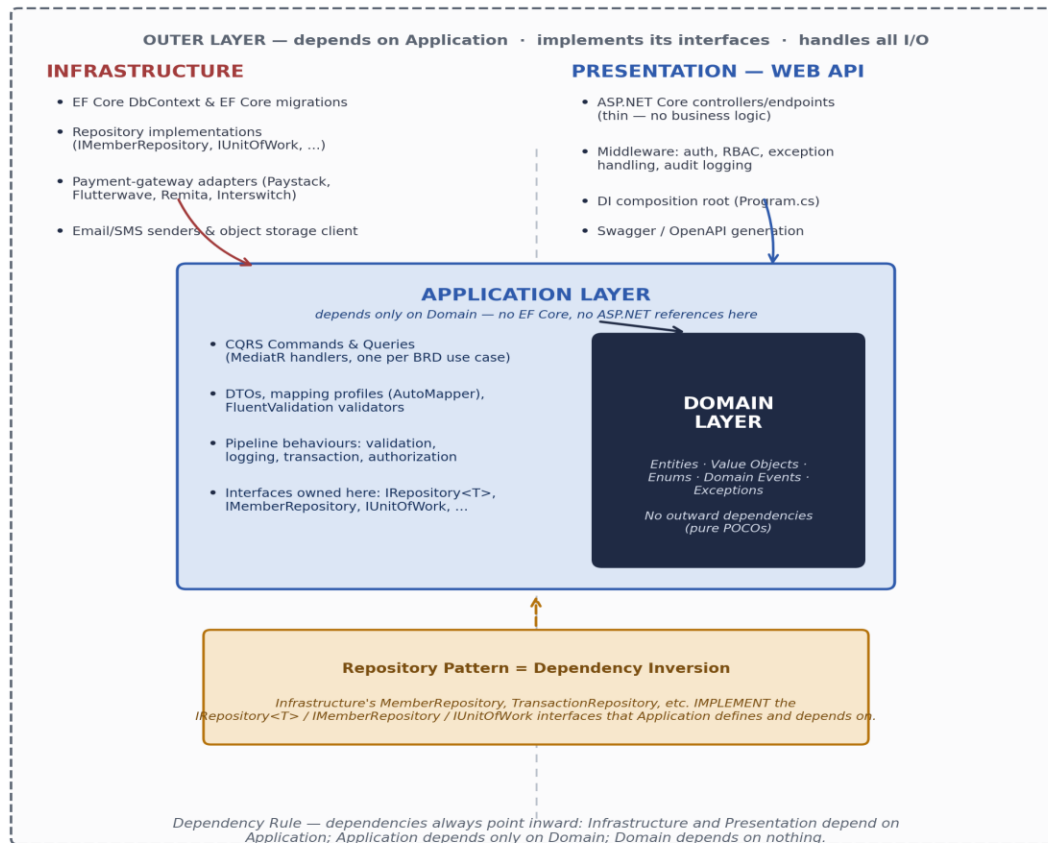
The system is built using **Clean Architecture** (the layering popularised by Robert C. Martin and the pattern behind Microsoft's own eShopOnWeb / Jason Taylor .NET templates), implemented with the **Repository Pattern** and **Unit of Work** over Entity Framework Core. This supersedes the generic layered/modular-monolith framing of the initial draft of this document — the module boundaries described there (Membership, Financial, Reporting) still hold, but they are now expressed as Clean Architecture layers rather than a bespoke six-layer stack.

2.1 Why Clean Architecture for This System

- **Business logic outlives any one framework.** Approval routing, ledger-posting rules, duplicate-member detection and financial-clearance calculation are the parts of this system that are actually valuable and BRD-specific. Clean Architecture keeps every one of them in the Domain and Application layers, written as plain C#, with no dependency on ASP.NET Core or Entity Framework Core. Upgrading .NET versions, changing ORMs, or replacing the web framework never touches this code.
- **Testability without a database.** Because Application-layer handlers depend on repository interfaces, not EF Core, the ledger-posting and approval-routing logic — the highest-risk logic in the system — can be unit tested with in-memory fakes or mocks, fast and without a test database. Section 14 builds the testing strategy directly on this property.
- **The existing system's logic has one clear destination.** Every piece of business logic recovered from the audit of the existing 60%-complete system has an unambiguous home: a rule about a national number is a Domain invariant; a duplicate-detection algorithm is an Application query handler; nothing needs to be reverse-engineered into an architecture that doesn't have a place for it.
- **Database Architects get a stable contract.** The Application layer depends on repository interfaces it defines, not on EF Core directly — so schema and query optimisation work in Infrastructure (Section 8, Section 4.3) can proceed without the Application layer's business logic changing, as long as the interface contract is honoured.

2.2 The Dependency Rule

Source-code dependencies point in one direction only: inward. Infrastructure and Presentation depend on Application; Application depends only on Domain; Domain depends on nothing else in the system. Nothing in an inner layer may reference a type from an outer layer — not a using statement, not a NuGet package. This single rule is what Figure 2 visualises, and it is the rule code review should check first on any pull request that adds a new dependency.

Figure 2 — Clean Architecture: Layers & the Dependency Rule (.NET)*Figure 2 — Clean Architecture: the four layers and the Dependency Rule, as applied to this system.*

2.3 The Four Layers, at a Glance

- **Domain** — entities, value objects, enums, domain events and exceptions. Pure C# (POCOs); zero outward dependencies.
- **Application** — CQRS commands and queries (MediatR handlers), DTOs, validation, and the repository/service interfaces the rest of the system implements. Depends only on Domain.
- **Infrastructure** — EF Core, the repository and Unit of Work implementations, payment-gateway and notification adapters, identity. Implements Application's interfaces.
- **Presentation (Web API)** — ASP.NET Core controllers, middleware, the DI composition root. Thin — translates HTTP to Application commands/queries and back.

2.4 Technical Principles

- **Persistence ignorance.** Domain entities never reference EF Core, System.Data, or any Infrastructure type. A Domain class should compile in a project with no NuGet packages at all.

- **The Application layer owns its interfaces.** IMemberRepository, IUnitOfWork, IPaymentGatewayAdapter and similar are defined where they are consumed (Application), not where they are implemented (Infrastructure) — this is the Dependency Inversion Principle applied deliberately, and it is what Section 5 is about.
- **One command or query per use case.** Every BRD functional requirement that changes or reads state maps to a named Application-layer command or query (e.g. CreateFinancialTransactionCommand, GetMemberFinancialClearanceQuery) — not a generic "service" method with optional parameters.
- **Controllers do not contain business logic.** A controller action's job is: map the request to a command/query, call IMediator.Send(...), map the result to a response. Any if-statement that encodes a business rule inside a controller is a defect.
- **Configurable over hard-coded.** Approval chains, thresholds, report schedules and role permissions are data the Application and Infrastructure layers read, not values compiled into the code.

3. Solution and Project Structure

The .NET solution is organised as one project per architectural layer, plus one test project per layer that needs one. This is the structure to scaffold in Phase 1 (Section 16.1) before any feature work begins.

Project	Layer	Depends On	Contains
KSM.Domain	Domain	(nothing)	Entities, value objects, enums, domain events, domain exceptions
KSM.Application	Application	KSM.Domain	CQRS commands/queries and handlers (MediatR), DTOs, AutoMapper profiles, FluentValidation validators, pipeline behaviours, repository/service interfaces
KSM.Infrastructure	Infrastructure	KSM.Application, KSM.Domain	EF Core DbContext and migrations, repository and Unit of Work implementations, payment/notification adapters, Identity
KSM.WebApi	Presentation	KSM.Application, KSM.Infrastructure, KSM.Domain*	ASP.NET Core controllers, middleware, Program.cs composition root, Swagger config
KSM.Application.Tests	Test	KSM.Application, KSM.Domain	Unit tests for handlers, validators and domain logic, using fake/mock repositories
KSM.Infrastructure.Tests	Test	KSM.Infrastructure	Repository/EF Core integration tests, typically against Testcontainers-provisioned MySQL
KSM.WebApi.Tests	Test	KSM.WebApi	API integration tests via WebApplicationFactory — full pipeline, in-memory or containerised test database

* *KSM.WebApi references KSM.Domain only for the composition root's DI registrations; no controller or middleware should use a Domain type directly where an Application DTO is the correct contract.*

3.1 Reference Rules — Enforced, Not Just Documented

- KSM.Domain has no project references and no third-party NuGet packages beyond what is strictly necessary for language features (e.g. nothing beyond the BCL).
- KSM.Application references KSM.Domain only. It must never reference Microsoft.EntityFrameworkCore, Microsoft.AspNetCore.*, or KSM.Infrastructure.
- KSM.Infrastructure references KSM.Application and KSM.Domain, and implements every interface KSM.Application declares that needs an external concern (database, gateway, email/SMS, clock, current-user).
- KSM.WebApi is the only project allowed to reference KSM.Infrastructure directly — and only inside Program.cs, to wire up dependency injection. No controller resolves an Infrastructure type directly; it depends on IMediator and Application-layer contracts.
- **Recommendation:** add a architecture/dependency test (e.g. using NetArchTest.Rules or ArchUnitNET) to KSM.Application.Tests that fails the build if KSM.Application ever references EF Core or ASP.NET Core — this turns the Dependency Rule from a code-review convention into something CI enforces automatically.

3.2 Illustrative Folder Layout

KSM.Domain/	
Entities/	Member, Transaction, JournalEntry, Budget, ...
ValueObjects/	NationalNumber, Money, ApprovalThreshold, ...
Enums/	MemberStatus, TransactionType, ApprovalStage, ...
Events/	MemberStatusChangedEvent, PaymentPostedEvent, ...
Exceptions/	DomainException and derivatives
KSM.Application/	
Membership/Commands/	CreateMemberCommand, ChangeMemberStatusCommand, ...
Membership/Queries/	GetMemberByIdQuery, SearchMembersQuery, ...
Financial/Commands/	CreateFinancialTransactionCommand,
ApproveTransactionCommand, ...	
Financial/Queries/	GetLedgerQuery, GetMemberClearanceQuery, ...
Reporting/Queries/	GetDashboardQuery, GetOutstandingDuesReportQuery,
...	
Common/Interfaces/	IRepository<T>, IMemberRepository, IUnitOfWork,
	IPaymentGatewayAdapter, INotificationService,
ICurrentUserService	
Common/Behaviours/	ValidationBehaviour, LoggingBehaviour,
TransactionBehaviour	
KSM.Infrastructure/	
Persistence/	KsmDbContext, EF Core configurations, Migrations/

Repositories/ UnitOfWork, ...	MemberRepository, TransactionRepository,
Payments/	PaystackAdapter, FlutterwaveAdapter, ...
Notifications/	EmailSender (MailKit), SmsSender
Identity/	token issuance, ASP.NET Core Identity integration
KSM.WebApi/ Controllers/ ReportsController, ...	MembersController, TransactionsController,
Middleware/ AuditLoggingMiddleware	ExceptionHandlingMiddleware,
Program.cs	composition root – DI registration for all four layers

5. Repository Pattern and Unit of Work

the Repository Pattern is the mechanism by which the Application layer reaches persisted data without depending on Entity Framework Core. It is the concrete instance of the Dependency Inversion Principle that makes the Dependency Rule (Section 2.2) achievable in practice, and it is not optional or a "nice to have" — every access to the database from Application-layer code goes through a repository interface.

5.1 The Pattern, Concretely

Application defines the contract. Infrastructure implements it. A handler in KSM.Application never sees DbContext, DbSet<T>, or an EF Core LINQ provider — only the interface.

```
// KSM.Application/Common/Interfaces/IRepository.cs
public interface IRepository<T> where T : class
{
    Task<T?> GetByIdAsync(Guid id, CancellationToken ct);
    Task<IReadOnlyList<T>> ListAsync(ISpecification<T> spec,
    CancellationToken ct);
    Task AddAsync(T entity, CancellationToken ct);
    void Update(T entity);
    void Remove(T entity);
}

// KSM.Application/Common/Interfaces/IMemberRepository.cs
public interface IMemberRepository : IRepository<Member>
{
    Task<Member?> GetByNationalNumberAsync(string nationalNumber,
    CancellationToken ct);
    Task<bool> IsDuplicateAsync(string name, DateOnly dateOfBirth,
    CancellationToken ct);
}

// KSM.Application/Common/Interfaces/IUnitOfWork.cs
public interface IUnitOfWork
```

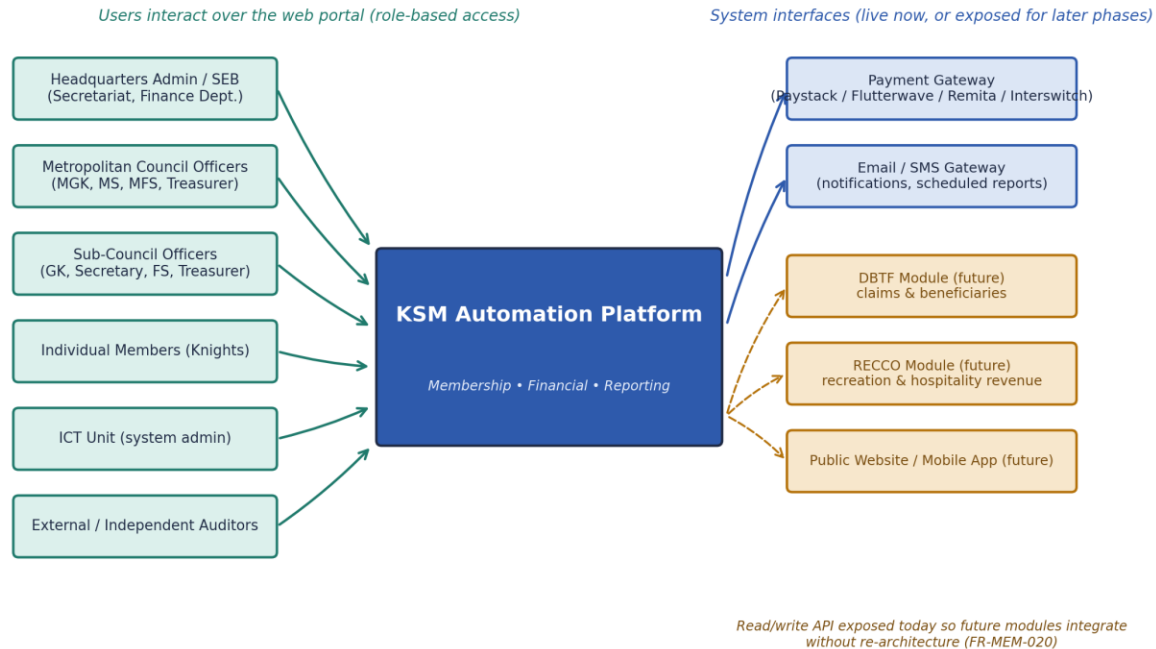
```
{  
    Task<int> CommitAsync(CancellationTokentoken ct);  
}
```

Infrastructure implements both against EF Core:

```
// KSM.Infrastructure/Repositories/MemberRepository.cs  
public class MemberRepository : IMemberRepository  
{  
    private readonly KsmDbContext _db;  
    public MemberRepository(KsmDbContext db) => _db = db;  
  
    public async Task<Member?> GetByNationalNumberAsync(string n,  
CancellationTokentoken ct) =>  
        await _db.Members.SingleOrDefaultAsync(m => m.NationalNumber == n,  
ct);  
    // ... remaining IRepository<Member> / IMemberRepository members  
}
```

7. System Context

The platform sits between KSM's officers and members on one side, and a small set of external systems on the other: a payment gateway and an email/SMS gateway today. DBTF, RECCO, the public website and a future mobile app are explicitly out of scope for this build, but the Presentation layer's API (Section 11.4) is designed so they can integrate later without any change to Domain, Application or Infrastructure.

Figure 1 — System Context Diagram*Figure 1 — System Context Diagram: actors, the platform, and its external and future integrations.***Actors**

- **Headquarters Admin / SEB / Finance Department** — organisation-wide administration, financial policy, and governance sign-off.
- **Metropolitan Council Officers (MGK, MS, MFS, Treasurer)** — regional administration and financial oversight of their Sub-Councils.
- **Sub-Council Officers (GK, Secretary, FS, Treasurer)** — front-line data capture: membership, attendance, dues and expenses.
- **Individual Members (Knights)** — self-service access to their own profile, financial standing and attendance history.
- **ICT Unit** — technical administration and first-line support after handover.

8. Data Architecture**8.1 Schema Strategy**

One physical database, logically separated into schema groups by table prefix or MySQL schema — **membership_***, **finance_***, **reporting_*** (read-models only), and a shared **core_*** group for users, roles, permissions, audit_log, notifications and documents. Every table is reached exclusively through the

Infrastructure-layer repository that owns it (Section 5) — there is no direct DbContext access from Application or Presentation code, and no cross-module table access that bypasses a repository interface.

Schema changes are made only through EF Core migrations generated from changes to the entity type configurations in KSM.Infrastructure/Persistence — never a manual ALTER TABLE against any environment, including production. Every migration is reviewed the same way application code is, and is applied through the deployment pipeline (Section 16), not run ad hoc by a developer with production credentials.

8.2 Core Entities (Domain Layer)

The table below is the logical data model Database Architects should design the physical schema against, and that Domain-layer developers should model as entities/value objects; it matches the BRD's Data Requirements (#8) entity list not limited to the following;

Entity	Owning Module	Key Attributes (illustrative)
Member	Membership	National number (unique, immutable, never reassigned), name, DOB, contact, Sub-Council, Metro Council, degree, status, admission date, photo
Family / Next-of-Kin	Membership	Member reference, relationship, name, contact details
Sub-Council / Metro Council	Membership	Name, code, parent unit reference, location, officers
Degree Record	Membership	Member reference, degree level, exam date, result, certifying body
Status Change	Membership	Member reference, previous/new status, effective date, reason, approver
Chart of Accounts	Financial	Account code, name, type, parent account, unit applicability
Journal Entry	Financial	Date, unit, debit/credit account, amount, narration, posted-by, approval reference
Transaction (income/expense)	Financial	Type, category, unit, amount, date, requester, approval-chain status, supporting documents
Budget	Financial	Unit, period, department, line items, approved amount, actual-to-date
Payment	Financial	Payer/member reference, amount, method, gateway reference, receipt number, status
Report Definition	Reporting	Report type, scope/unit, filters, schedule, recipients
User Account	Core / shared	Username, role, unit assignment, credentials, MFA status, last login

Entity	Owning Module	Key Attributes (illustrative)
Audit Log	Core / shared	User, action, entity affected, before/after values, timestamp, IP address

8.3 Data Integrity Rules — Enforced in the Domain and Application Layers

- A member's national number is generated by the system, unique, and never reused — even after death or expulsion. Enforced as a Domain invariant on Member plus a unique database constraint as the defence-in-depth backstop.
- A status of "Deceased" or "Expelled" requires a corresponding effective date and approver on record before it is accepted — validated by `ChangeMemberStatusCommandValidator` (`FluentValidation`) before the handler runs, and re-checked as a Domain invariant inside `Member.ChangeStatus(...)`.
- A member transfer between Sub-Councils updates both the losing and receiving Sub-Council's rolls atomically — one `IUnitOfWork.CommitAsync()` call inside `TransferMemberCommandHandler`, never two independent saves.
- Every financial posting is wrapped in a Unit of Work transaction so it is either fully applied or fully rolled back; a reversed or refunded payment retains the original transaction record rather than being deleted or edited in place — enforced by never exposing an Update or Remove path for posted Transaction entities, only a compensating-entry command.
- A member's "financial good standing" flag is a derived/cached value, recomputed by a domain-event handler whenever the underlying dues ledger changes (`PaymentPostedEvent` → recompute) — it is never written to directly by any command.
- Reports at every hierarchy level only ever query data scoped to that unit and its subordinate units — enforced inside the Reporting query handlers via `ICurrentUserService`'s unit scope, not left to the UI to hide unauthorised rows.

8.6 Data Retention

Data	Retention
Financial records (ledger, transactions, budgets)	Minimum 7 years, per standard accounting practice
Membership records	Indefinite
Audit log	Minimum 12 months online; recommend archiving rather than deleting thereafter
Generated report snapshots	Minimum 12 months
Backups	Daily (short-term), weekly (medium-term), monthly (long-term, retained 12 months)

9. Cross-Cutting Concerns

RBAC, audit logging, validation and the approval workflow are needed by every module. Clean Architecture's answer to "how do we avoid building this three times" is the same in every case: implement it once as an Application-layer MediatR pipeline behaviour (or a piece of shared middleware at the Presentation edge), and every command/query gets it automatically by flowing through `IMediator.Send(...)` — there is no code path that bypasses the pipeline.

9.1 MediatR Pipeline Behaviours

```
services.AddMediatR(cfg => {
    cfg.RegisterServicesFromAssembly(typeof(CreateMemberCommand).Assembly);
    cfg.AddOpenBehavior(typeof(ValidationBehaviour<, >));
    cfg.AddOpenBehavior(typeof(AuthorizationBehaviour<, >));
    cfg.AddOpenBehavior(typeof(LoggingBehaviour<, >));
    cfg.AddOpenBehavior(typeof(TransactionBehaviour<, >));
});
```

- **ValidationBehaviour** runs every registered `FluentValidation` validator for the request before the handler executes; a failed validation short-circuits the pipeline with a structured error, and the handler never runs against invalid input.
- **AuthorizationBehaviour** checks the current user (`ICurrentUserService`) against the permissions the command/query declares (an attribute or marker interface, e.g. `IRequireRole("FinancialSecretary")`) — this is where RBAC (Section 10.1) is actually enforced, once, for every request regardless of which controller it came from.
- **LoggingBehaviour** and the `AuditLoggingMiddleware` together record every create/update/delete/approval action — actor, timestamp, entity, before/after values (Section 10.4) — without each handler writing its own logging code.
- **TransactionBehaviour** wraps command handlers in `IUnitOfWork.CommitAsync()`, so a handler that touches multiple repositories either commits everything or nothing (Section 8.3), without every handler re-implementing transaction management.

9.2 Approval Workflow

The BRD independently asks for configurable, multi-step approval in two places — member status changes (FR-MEM-008) and financial transactions/budgets (FR-FIN-025, FR-FIN-028–030). Both are modelled as an `ApprovalChain` value object/entity in the Domain layer plus a generic `RouteForApprovalCommand` / `ApproveStepCommand` pair in Application, configured per module (approver roles, thresholds) rather than duplicated as separate approval logic in Membership and Financial. Segregation of duties — the person who raises a transaction cannot also give its final approval — is enforced as a Domain invariant on the approval chain, not a UI convention.

9.3 Notification and Document Handling

`INotificationService` and a document-storage interface are Application-layer contracts, implemented in Infrastructure (Section 4.3, Section 11.3), consumed by domain-event handlers — e.g. a

PaymentPostedEvent handler in Application sends a receipt notification, without the Financial command handler that raised the payment needing to know how notifications are delivered.

10. Security Architecture

The BRD names role-based access control, encryption, backups, audit logs and multi-factor authentication explicitly (#16). This section states how each is implemented within the layers defined in Section 4.

10.1 Authentication and Authorisation

- **RBAC as data, evaluated in the Application pipeline.** [Architecture Decision] The permissions matrix (BRD #10 — Super Administrator, Headquarters Administrator, Headquarters Financial Officer, Headquarters Auditor, SEB Member, MGK/Metro Administrator, MS, MFS, Metro Treasurer, GK/Sub-Council Administrator, Sub-Council Secretary/FS/Treasurer, Data Entry Officer, DBTF/RECCO integration users, Independent Auditor, Member, Guest, ICT Unit) is stored as configurable role/permission records and evaluated by AuthorizationBehaviour (Section 9.1) on every command and query. Adding a role or adjusting what it can see is a configuration change, not a code deployment or a new if-statement in a controller.
- **Multi-factor authentication** is mandatory for the Headquarters Administrator, Headquarters Financial Officer and Auditor roles (BRD-explicit), implemented via ASP.NET Core Identity's MFA support, and available to all other roles.
- **Session management.** Token-based authentication (JWT) with a configurable expiry and refresh-token rotation; sessions time out automatically after a configurable period of inactivity.

10.2 Data Protection

- TLS 1.2+ enforced on every endpoint, terminated at the Nginx edge layer in front of Kestrel.
- AES-256 (or equivalent) encryption at rest for sensitive fields — member personal data and financial records in particular — and for backups.
- Password policy enforced by ASP.NET Core Identity: minimum complexity rules, forced periodic rotation for privileged accounts, and lockout after repeated failed attempts.

10.3 Application Security (OWASP Top 10)

Risk	Mitigation
SQL injection	EF Core parameterised queries exclusively, reached only through repositories (Section 5) — no raw, string-concatenated SQL anywhere in Infrastructure
Cross-site scripting (XSS)	Output encoding in the React portal; a Content-Security-Policy header set by ASP.NET Core middleware
Cross-site request forgery (CSRF)	Anti-forgery tokens on all state-changing requests from the web portal; SameSite cookie policy

Risk	Mitigation
Broken access control	AuthorizationBehaviour (Section 9.1) — a single, centrally-enforced pipeline stage, never a per-controller ad-hoc check
Security misconfiguration	Hardened HTTP response headers via ASP.NET Core middleware, no default credentials, NuGet dependency vulnerability scanning in CI
Sensitive data exposure	Encryption in transit and at rest (Section 10.2); least-privilege database accounts per environment
Insufficient logging and monitoring	LoggingBehaviour and AuditLoggingMiddleware (Section 9.1) plus infrastructure monitoring (Section 12)

10.4 Audit Logging

Every create, update, delete and approval action — in any module — is written to an append-only audit log capturing the acting user, timestamp, the entity affected, and before/after values, via LoggingBehaviour and AuditLoggingMiddleware (Section 9.1). Financial postings are never edited or deleted after the fact; a correction is a new, linked reversal/adjustment command (Section 8.3), so the audit trail and the ledger always agree.

10.5 Segregation of Duties

The approval-chain Domain invariant (Section 9.2) enforces, as a system rule rather than a UI convention, that the person who raises a financial transaction cannot also be the one who gives its final approval. Officer-level accounts should be reviewed periodically so access is deactivated promptly when a person's tenure in a role ends.

11. Integration Architecture

11.1 API Design

All access to the system — including from the React portal itself — goes through one versioned REST API exposed by KSM.WebApi (e.g. /api/v1/...), documented with OpenAPI/Swagger generated from the controllers (Section 4.4). There is no back-door query path into the database that a future integration would need to be rebuilt around: every consumer, present or future, calls the same controllers, which call the same Application-layer commands and queries.

11.2 Payment Gateway Adapter

[Architecture Decision] IPaymentGatewayAdapter is defined once in KSM.Application (Section 4.2) — InitiateAsync, VerifyAsync, HandleWebhookAsync, RefundAsync — and implemented once per provider in KSM.Infrastructure (PaystackAdapter, FlutterwaveAdapter, RemitaAdapter, InterswitchAdapter), so Financial-module command handlers never talk to a specific provider's SDK or REST API directly. This directly satisfies the BRD's requirement that the provider be selected competitively and be changeable later without re-architecture — swapping providers means writing one new Infrastructure-layer class, not touching Application or Domain.

- Funds settle to a Headquarters-controlled bank account; the split across dues/levy/Headquarters share happens inside RecordPaymentCommandHandler, not via separate bank-level transfers.
- Every gateway transaction reference is stored against its corresponding Payment entity, to support reconciliation.
- Webhooks are handled idempotently inside the adapter's HandleWebhookAsync implementation — a duplicate or out-of-order webhook delivery must never double-post a payment; a scheduled reconciliation job (Hangfire/Quartz.NET) periodically checks gateway status against local records for anything left pending.
- Failed or pending gateway transactions are clearly distinguished from successful ones and never affect a member's financial clearance status.
- Gateway transaction fees are recorded as a distinct, reportable expense line.

11.3 Notification Adapter

Email and SMS delivery are likewise behind a single INotificationService interface (Section 4.2), implemented by an EmailSender (MailKit) and SmsSender in Infrastructure (Section 4.3), so the underlying SMTP relay or SMS aggregator can be changed by replacing one Infrastructure class — never by touching the domain-event handlers (Section 9.3) that trigger notifications.

11.4 Future Integration API (DBTF, RECCO, Website, Mobile App)

FR-MEM-020 asks for a read API so the future DBTF and RECCO modules can query verified membership status without duplicating the record. Because every access path already runs through the same versioned Web API and Application-layer queries, this is a matter of exposing a scoped, rate-limited subset of existing read endpoints — read-only for status/clearance queries, authenticated by unit-

specific API keys (the DBTF Officer and RECCO Officer roles already appear in the permissions matrix, BRD #10) — rather than building a parallel integration surface. Nothing about the Domain, Application or Infrastructure layers changes to support it.

12. Deployment / Infrastructure Architecture

The BRD's own infrastructure recommendation (#15) is the basis for the production environment: a single mid-sized VPS or small cloud instance, sized for growth rather than over-provisioned on day one, hosted with a provider offering predictable pricing and Nigeria-reachable latency, running the ASP.NET Core Web API on Kestrel.

Figure 4 — Deployment / Infrastructure Architecture

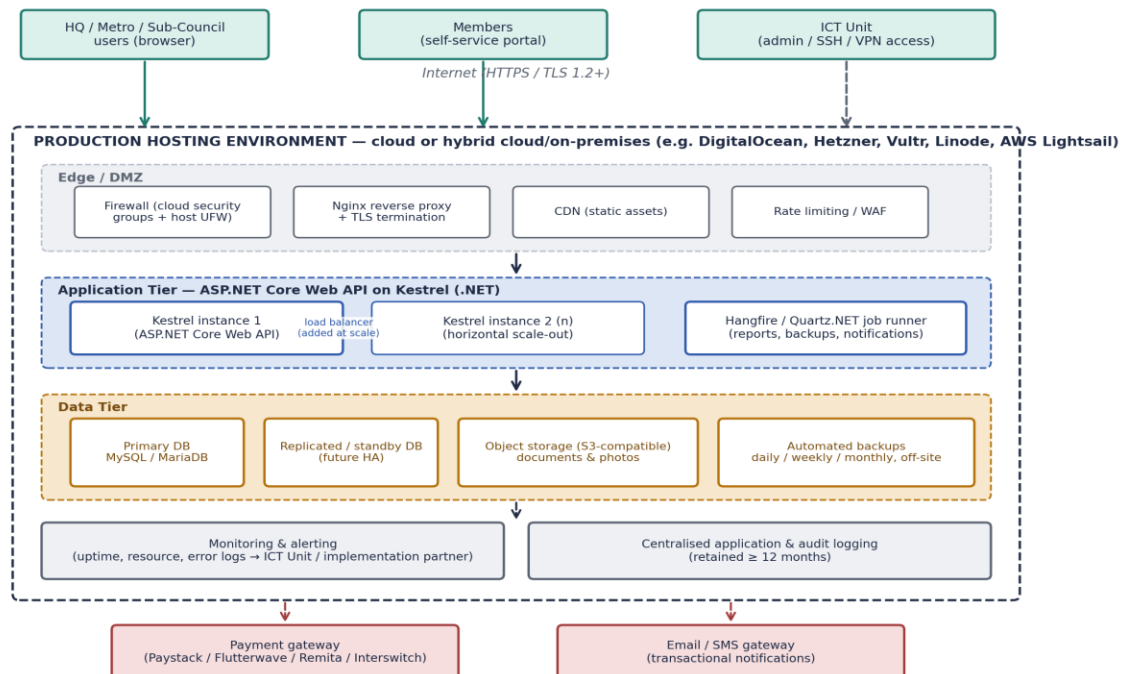


Figure 4 — Deployment / Infrastructure Architecture: edge, application, and data tiers within the production hosting environment, plus external integrations.

12.1 Environment Strategy

Environment	Purpose	Notes
Development	Individual developer work and integration of in-progress features	Local or shared dev server; seeded with synthetic data only

Environment	Purpose	Notes
Staging / QA	Formal testing against the BRD's Acceptance Criteria (#21) before any release	Mirrors production configuration; used for UAT sign-off (Section 14)
Pilot	The BRD's phased rollout (#17): a small number of Sub-Councils plus one Metro Council, running in parallel with manual processes for at least one full dues cycle	Production infrastructure, real (initially limited) data
Production	National rollout after pilot sign-off	Full sizing per Section 12.2; the environment shown in Figure 4

12.2 Sizing (BRD #15, restated for the development team)

Specification	Recommendation
CPU	4–8 vCPUs, scalable to 16 as usage grows
RAM	8–16 GB, scalable to 32 GB
Storage	160–320 GB NVMe SSD, plus an object-storage add-on for documents/photos
Bandwidth	Minimum 4–5 TB/month transfer allowance
Backups	Daily automated snapshot, weekly full backup, monthly archive retained 12 months
Suggested providers	DigitalOcean, Hetzner, Vultr, Linode, or AWS Lightsail

Final sizing should be confirmed against actual concurrent-user and data-volume estimates gathered during Phase 1 technical discovery (Section 16.1); the figures above are a budgeting starting point, not a hard cap — Kestrel instances are stateless (Section 15), so additional instances can be added behind a load balancer once a single instance is no longer sufficient.

12.3 Backup, Restore and Disaster Recovery

- Recovery Point Objective (RPO): 24 hours. Recovery Time Objective (RTO): 8 hours — both BRD-mandated targets the backup schedule and restore procedure must be tested against.
- Backups are stored off-site (not just on the same host) and encrypted at rest.
- A full restoration drill is run quarterly, and its result is documented — an untested backup is not a backup.
- The migration pre-cutover backup (Section 8.5) is retained indefinitely, separately from the routine backup rotation.

13. Key Workflow Architecture

Figure 3 (Section 5) already traced a single request's path through the four layers in the abstract. This section traces the BRD's own workflows (#7) through that same path, naming which project and which command/query owns each step, using the online-payment-with-approval workflow as the fullest illustration.

Figure 5 — Illustrative Workflow: Online Payment → Approval → Clearance → Reporting

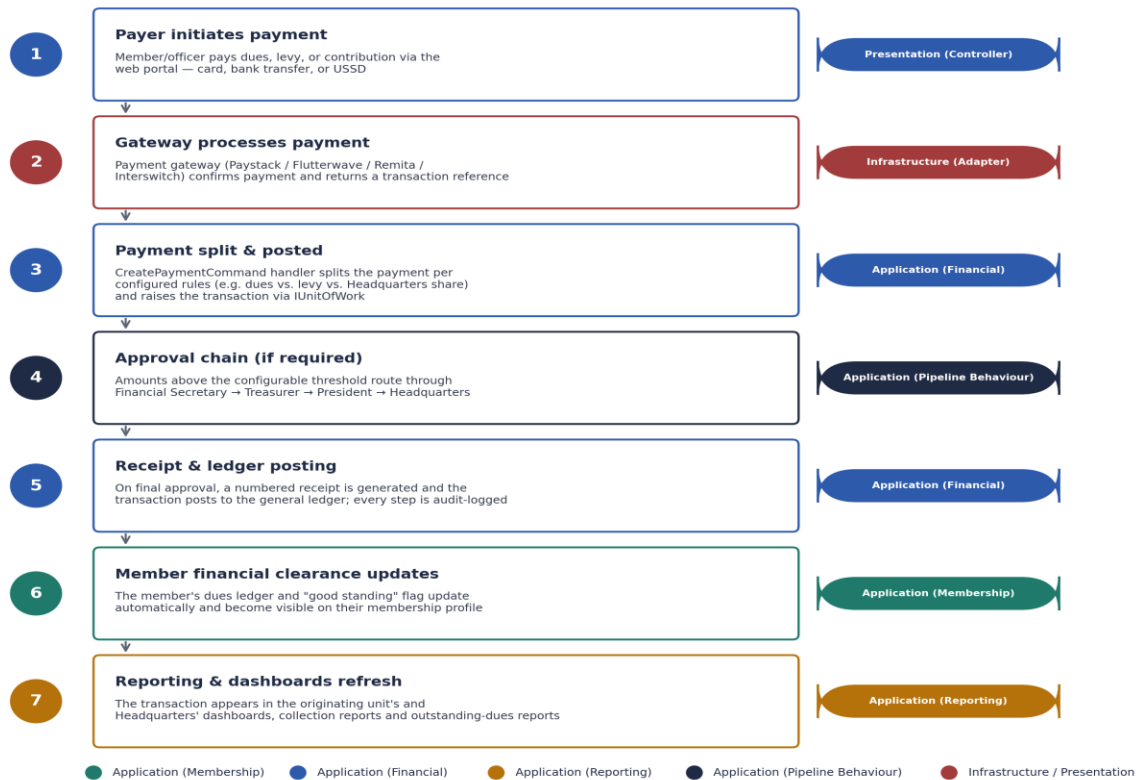


Figure 5 — Illustrative Workflow: Online Payment → Approval → Clearance → Reporting, mapped to the owning layer at each step.

Other BRD Workflows, and Where They Live

- **Member recruitment and admission (BRD #7).** Forms 1–4/6–7 capture is a CreateMemberCommand from KSM.WebApi's MembersController; duplicate/completeness validation runs in CreateMemberCommandValidator and Member's Domain invariants; national-number generation is a Domain service; the admission approval step runs through the shared approval workflow (Section 9.2); the resulting nominal-roll update and the Financial Module's "open a dues account" notification both fire from the same MemberAdmittedEvent domain event.
- **Member status change — suspension, expulsion, transfer, death (BRD #7).** ChangeMemberStatusCommand routes through the approval workflow to the correct approver

role for that change type (e.g. Metro President for suspension, Headquarters for expulsion) and, on approval, updates status, nominal rolls, dues billing and reporting atomically in one `IUnitOfWork.CommitAsync()` call. `TransferMemberCommand` additionally updates both the losing and receiving Sub-Council's rolls in that same transaction (Section 8.3).

- **Financial transaction and approval (BRD #7).** This is the general case of steps 3–5 in Figure 5, without the online-payment-specific steps 1–2: `CreateFinancialTransactionCommand` routes through a Financial Secretary/Treasurer/President chain, escalating to Headquarters automatically when a transaction exceeds its configured threshold or is flagged sensitive.

14. Testing Strategy

Clean Architecture is built to be tested — this section exists to make that concrete for Testers, not just developers. Because the Application layer depends on interfaces, not concrete infrastructure, the highest-value tests in this system (does the ledger post correctly, does an approval route to the right person, does a duplicate member get caught) run without a real database and in milliseconds. Testers should expect three distinct kinds of automated test, mapped directly to the layers in Section 4, plus manual/UAT testing against the BRD's Acceptance Criteria (#21).

14.1 Test Projects and What Each One Covers

Test Project	Targets	Typical Technique	Example
KSM.Application.Tests	Command/query handlers, validators, domain logic	Unit tests with in-memory fakes or mocks (e.g. Moq/NSubstitute) for every repository/service interface — no database	CreateFinancialTransactionCommandHandler posts correctly given a mocked ITransactionRepository and IUnitOfWork
KSM.Infrastructure.Tests	Repository and EF Core mapping correctness, migrations	Integration tests against a real, disposable database — typically MySQL provisioned via Testcontainers per test run	MemberRepository.GetByNationalNumberAsync returns the correct entity against a seeded Testcontainers MySQL instance
KSM.WebApi.Tests	Controllers, middleware, the full request pipeline, RBAC enforcement	API integration tests via WebApplicationFactory<Program> — real HTTP pipeline, test or in-memory database	POST /api/v1/transactions returns 403 for a role without the FinancialSecretary permission; returns 201 and posts the ledger entry for one that has it
End-to-end (Playwright/Cypress)	Critical user journeys through the React portal against a full staging deployment	Browser-driven, black-box	A Sub-Council Treasurer logs in, records a payment, and sees it reflected in the member's financial clearance status

14.2 What to Test at Each Layer

- **Domain.** Entity invariants and value-object behaviour in isolation — e.g. Member.ChangeStatus(...) throws InvalidStatusTransitionException for an illegal transition; Money refuses to add two different currencies. No mocks needed at all; these are the fastest, simplest tests in the suite.

- **Application.** Given a mocked repository/service, does the handler produce the right result and call the right repository methods? Does the validator reject the inputs the BRD's business rules say are invalid? Does the pipeline's AuthorizationBehaviour reject a request from a role without permission? This is where most of the BRD's functional requirements (FR-MEM-xxx, FR-FIN-xxx, FR-REP-xxx) get their primary automated coverage.
- **Infrastructure.** Does the repository's EF Core query actually return the right rows against a real database engine (LINQ-to-SQL translation bugs do not show up against an in-memory provider — always test repositories against MySQL, never EF Core's InMemory provider, for anything going to production)? Do migrations apply cleanly to a fresh database and to a copy of a populated one?
- **Presentation.** Does the controller map requests/responses correctly, return the right HTTP status codes, and is every endpoint actually reachable only by the roles the permissions matrix says it should be? WebApplicationFactory-based tests are the right tool — they exercise the real middleware pipeline, including AuthorizationBehaviour and ExceptionHandlingMiddleware.

14.3 Mocking Strategy — Why Testers Should Care About the Repository Pattern

Because Application-layer handlers depend only on repository interfaces (Section 5), a test can substitute a fake implementation that holds data in memory, with no database, no test fixtures to seed and tear down, and no flakiness from shared test infrastructure. This is the direct, practical payoff of the architecture for the test suite: a Tester writing an Application-layer test for "a transaction above the threshold must route to Headquarters" configures a fake `ITransactionRepository`, calls the handler, and asserts on what it returns and what it tried to save — nothing about the database needs to exist for that test to be meaningful.

14.4 UAT and BRD Acceptance Criteria

Automated tests verify the architecture is correctly built; User Acceptance Testing against the BRD's Acceptance Criteria (#21) verifies the system does what KSM's officers actually need. Testers should track both explicitly and not treat one as a substitute for the other:

- **Membership:** gap-analysis items (Section 16.2) closed and tested; DBTF/Secretariat reconciliation (Section 8.5) complete and signed off; zero critical or high-severity defects open.
- **Financial:** the full transaction lifecycle (request → approval → posting → reporting) tested end-to-end for income, expense and budget; trial balance reconciles to zero variance in UAT; payment gateway integration tested with live test transactions against each adapter (Section 11.2).
- **Reporting:** all standard reports produce correct, hierarchy-scoped results verified against manually calculated control figures; export to Excel/PDF/CSV verified; at least one scheduled report successfully delivered by email in testing.
- **Cross-cutting:** the roles/permissions matrix implemented and verified by role-based test accounts (Section 10.1); security requirements independently verified (Section 10.3); backup and restore successfully demonstrated (Section 12.3).

14.5 CI Gate

KSM.Application.Tests and KSM.WebApi.Tests run on every pull request; KSM.Infrastructure.Tests run at least on every merge to the main branch (Testcontainers adds runtime, so it may be excluded from the fastest per-commit loop by team agreement). A pull request that reduces coverage on Application-layer command/query handlers, or that adds a new Application-layer dependency on EF Core or ASP.NET Core (Section 3.1), should fail review.

16. Development Roadmap and Build Order

This section answers the question the rest of the document sets up: **concretely, in what order does the development team build this?** Clean Architecture implies a build order, not just a file layout: **Domain, then Application, then Infrastructure, then Presentation**, per BRD module, folded into the BRD's five implementation phases (#17).

16.1 Before Any Feature Work — Inception Tasks

1. **Technical audit of the existing system.** Work through the Gap Analysis Checklist (Section 16.2) against the existing ~60%-complete Node.js system's actual codebase and database — not assumptions — to confirm exactly what business rules and data the new Domain and Application layers must reproduce. Produce a short written findings memo before committing to further estimates; this is the gate the BRD's own risk register (#18) calls out as highest-impact if skipped.
2. **Scaffold the solution.** Create the projects in Section 3, wire the reference rules (Section 3.1), and add the architecture test that fails the build on a Dependency Rule violation, before any feature branch is opened against it.
3. **Environments and pipeline.** Stand up version control, the four environments in Section 12.1, and a CI/CD pipeline (lint → build → test → publish → deploy) before writing feature code against them.
4. **Agree the schema.** Document an ERD covering the entity model in Section 8.2, reviewed with Database Architects and signed off before the first EF Core migration is written.
5. **Build the cross-cutting pipeline first.** The MediatR pipeline behaviours, the approval workflow, and the notification/document interfaces (Section 9) are dependencies of every module — build and unit-test them before Membership or Financial handlers are written against them, so no module improvises its own version in the meantime.
6. **Testing scaffolding.** The four test projects in Section 14.1 set up and running in CI before feature work begins, not retrofitted afterward.

16.3 Phase-by-Phase Technical Delivery

- **Phase 1 — Foundation and Infrastructure.** Provision the production environment against Figure 4 and the sizing table (Section 12.2); configure Nginx/TLS/CDN/firewall; stand up CI/CD; run the technical audit (16.1.1); scaffold the four layers and their reference rules (Section 3); build and unit-test the shared pipeline behaviours (Section 9.1) and the approval workflow (Section 9.2).
- **Phase 2 — Membership Module.** Domain first (Member and related entities), then Application (commands/queries + validators), then Infrastructure (MemberRepository, EF Core configuration and migrations), then Presentation (MembersController). Work the checklist in Section 16.2 item by item; build the DBTF/Secretariat reconciliation tooling (Section 8.5) and run it to produce the signed-off migration reconciliation report — this is the gate before Phase 3 can integrate against a trustworthy member record.
- **Phase 3 — Financial Module.** Same order — Domain (ChartOfAccount, JournalEntry, Transaction, Budget, Payment and their invariants), Application (command/query handlers with double-entry and atomicity rules), Infrastructure (repositories, the first payment-gateway adapter — keep

IPaymentGatewayAdapter generic for the remaining three), Presentation. Wire the PaymentPostedEvent → financial-clearance handler back into Membership.

- **Phase 4 — Reporting Module and React Portal Rollout.** Build the reporting read-models and their refresh jobs (Section 8.4) before building query handlers against them; build the standard report library and the Excel/PDF/CSV export engine; build the four role-scoped React portal views against the versioned API; finalise and document the future-integration read API (Section 11.4), since Reporting's data shape informs it.
- **Phase 5 — Training, Hardening and Go-Live.** Independent security review against the OWASP checklist (Section 10.3); load/performance testing against the 200-concurrent-user and response-time targets (Section 15); a full backup-and-restore drill; pilot deployment to a small number of Sub-Councils plus one Metro Council; a parallel run of manual and system processes for at least one full dues cycle before manual registers are retired; officer training; national rollout.

18. Architecture Decision Log

A summary of the significant decisions made in this document, for quick reference during development and design review.

Decision	Chosen Approach	Why
Architecture style	Clean Architecture (Domain/Application/Infrastructure/Presentation) with the Repository Pattern and Unit of Work	Client-confirmed direction; isolates business logic from EF Core/ASP.NET Core, maximises testability, and gives Database Architects a stable contract to build behind
Technology stack	C# / .NET (ASP.NET Core Web API) backend; React preferred for the frontend, with a Node.js-based frontend as a fallback depending on staffing	Client-confirmed direction, superseding the BRD's assumption that the application server continues on Node.js
Deployment style	Modular monolith — one deployable KSM.WebApi, internally organised by Clean Architecture layers and BRD module folders, not microservices	BRD's sizing (200 concurrent users, single VPS, small team) does not justify microservices' operational overhead; layer and module boundaries are kept clean for later extraction
Database strategy	Single shared database, logically separated by schema, reached only through repositories	Eliminates cross-department record discrepancies the original ICT assessment identified; enables real-time financial clearance without a sync layer, while keeping a clean seam for schema evolution
Data access pattern	Repository Pattern + Unit of Work, interfaces owned by Application, implemented by Infrastructure	The Dependency Inversion Principle applied concretely; makes Application-layer logic unit-testable without a database (Section 14)
Cross-cutting concerns	MediatR pipeline behaviours (validation, authorization, logging, transaction), not per-handler code	One implementation per concern, applied uniformly to every command/query; no code path can bypass RBAC or audit logging

Decision	Chosen Approach	Why
Approval logic	One reusable, configurable approval-chain mechanism in Domain/Application	Both Membership status-changes and Financial transactions/budgets need multi-step configurable approval; one mechanism avoids duplicated, drifting logic
Payment integration	Gateway-agnostic IPaymentGatewayAdapter, one Infrastructure implementation per provider	BRD names four candidate PSPs and requires the choice to be competitive and changeable
Reporting data source	Dedicated read-models/materialised aggregates, queried read-only from Application	Protects transactional performance as volume grows, while still refreshing dashboards near-real-time; keeps Reporting's Application-layer queries simple
Existing system	Do not extend the Node.js codebase in place; rebuild on .NET per this architecture, informed by a mandatory technical audit of the existing system's logic and data	Client-directed technology change; the audit ensures no BRD-relevant business rule or data nuance already discovered is lost in the rebuild

19. Appendix

A. Requirement Traceability

Every component described in this document exists to satisfy one or more BRD functional requirements. Developers should reference the relevant requirement ID (FR-MEM-xxx, FR-FIN-xxx, FR-REP-xxx) in code comments, commit messages, command/query class names where practical, and test names, so a requirement can be traced forward to the code that implements it and a test that verifies it — and so a code change can be traced back to the business need that justifies it.

B. Glossary

Term	Meaning
KSM	Knights of St. Mulumba
SEB	Supreme Executive Board
DBTF	Death Benevolent Trust Fund
RECCO	KSM's recreation/hospitality operation
MGK / MS / MFS	Metro Grand Knight / Metro Secretary / Metro Financial Secretary
GK / FS	Grand Knight (Sub-Council head) / Financial Secretary (Sub-Council level)
BRD / SAD	Business Requirements Document / System Architecture Document
Clean Architecture	A layering style (Domain/Application/Infrastructure/Presentation) in which source-code dependencies point only inward, toward the Domain (Section 2)
Dependency Rule	The rule that an inner layer never references a type from an outer layer (Section 2.2)
Repository Pattern	An interface, owned by the Application layer, that abstracts data access; implemented by the Infrastructure layer (Section 5)
Unit of Work	An interface that commits one or more repository changes as a single atomic transaction (Section 5)
CQRS	Command Query Responsibility Segregation — separating state-changing operations (commands) from read operations (queries), implemented here via MediatR
MediatR	A .NET library implementing the mediator pattern, used here to dispatch commands/queries to their handlers and to run pipeline behaviours (Section 9.1)
DTO	Data Transfer Object — a flat object shape used to move data across a layer boundary, distinct from a Domain entity
POCO	Plain Old CLR Object — a class with no framework base class or attributes, as Domain entities are written (Section 4.1)

Term	Meaning
DI / IoC	Dependency Injection / Inversion of Control — how Infrastructure implementations are supplied to code that depends only on Application-layer interfaces (Section 4.4)
FluentValidation	A .NET validation library used for Application-layer command/query validators (Section 4.2)
Testcontainers	A library for running a real, disposable database (e.g. MySQL) inside a Docker container for integration tests (Section 14.1)
RBAC	Role-Based Access Control
COA	Chart of Accounts
NDPA	Nigeria Data Protection Act, 2023
API	Application Programming Interface — the versioned REST interface described in Section 11
ORM	Object-Relational Mapper — Entity Framework Core in this system
RPO / RTO	Recovery Point Objective / Recovery Time Objective — disaster-recovery targets (Section 12.3)
WAF / CDN	Web Application Firewall / Content Delivery Network
JWT / MFA	JSON Web Token / Multi-Factor Authentication
ADR	Architecture Decision Record — the format used in Section 18

C. Reference Documents

- Business Requirements Document — Membership, Financial Management and Reporting Modules, v1.0, July 2026, PN Consulting.
- KSM ICT Automation Feasibility Checklist (the underlying assessment the BRD was built from).